

SQL 语言

函数 Function

函数是存储在数据库中的一组SQL语句，可以通过名称调用它们。函数可以接受参数并返回一个值。函数通常用于封装常用的查询逻辑，以便在多个地方重复使用。

创建函数

```
create function dept_count (dept_name varchar(20))
returns integer
begin
declare d_count integer;
    select count (*) into d_count
    from instructor
    where instructor.dept_name = dept_name
return d_count;
end
```

调用函数

```
select dept_name, budget
from department
where dept_count (dept_name ) > 12
```

```
select *
from table (instructor_of ('Music'))
```

过程 Procedure

过程是存储在数据库中的一组SQL语句，可以通过名称调用它们。过程可以接受参数，但通常不返回值。过程通常用于执行一系列操作，例如插入、更新或删除数据。

创建过程

```
create procedure dept_count_proc (in dept_name varchar(20), out d_count
integer)
begin
    select count(*) into d_count
    from instructor
    where instructor.dept_name = dept_count_proc.dept_name
end
```

调用过程

```
declare d_count integer;  
call dept_count_proc( 'Physics', d_count);
```

通用编程语言

SQL支持的语法结构使其几乎具备通用编程语言的所有功能

复合语句

复合语句是一组SQL语句，可以作为一个单独的单元执行。复合语句通常用于存储过程和函数中，以便将多个操作组合在一起。

在 begin 和 end 之间包含多个 SQL 语句，可以声明局部变量。

控制结构

SQL 支持各种控制结构，例如条件语句（if-then-else）和循环语句（while、for）。这些控制结构允许程序根据不同的条件执行不同的操作。

```
if condition then  
    -- SQL 语句  
else  
    -- SQL 语句  
end if;
```

```
while condition loop  
    -- SQL 语句  
end loop;
```

```
for i in 1..10 loop  
    -- SQL 语句  
end loop;
```

声明外部语言过程和函数

SQL让我们能够用Java、C#、C或C++等编程语言定义函数。比SQL中定义的函数更高效，且无法在SQL中执行的计算可通过这些函数完成。

优点

在多数操作中效率更高，且表达能力更强。

缺点

- 实现该功能的代码可能需要加载到数据库系统中，并在数据库系统的地址空间内执行。
 - 数据库结构意外损坏的风险
 - 安全风险，允许用户访问未经授权的数据
- 存在替代方案，其安全性良好，但可能以性能下降为代价。
- 当效率高于安全性时，在数据库系统空间中直接执行。

解决安全问题

- 使用沙盒技术将外部语言代码与数据库系统的其余部分隔离开来。
- 在独立进程中运行外部语言函数/过程，且不访问数据库进程的内存。

上述两者都需要性能开销

递归查询

递归查询是指查询本身引用其自身的查询。SQL通过递归视图（也称为递归公共表表达式，CTE）来支持递归查询。

- 递归视图使得编写诸如传递闭包查询等无法通过递归或迭代实现的查询成为可能。
- 递归视图需要是单调的。

```
WITH RECURSIVE view_name (column1, column2, ...) AS (  
    -- 基本查询  
    SELECT ...  
    UNION ALL  
    -- 递归查询  
    SELECT ...  
    FROM view_name  
    WHERE ...  
)  
SELECT * FROM view_name;
```

Ranking Functions 排序函数

排名函数用于为查询结果集中的行分配唯一的排名值。常见的排名函数包括 RANK()、DENSE_RANK() 和 ROW_NUMBER()。

```
select ID, rank() over (order by GPA desc) as s_rank  
from student_grades
```

- RANK(): 为具有相同值的行分配相同的排名，并在排名之间留下间隙。
- 这里对GPA进行降序排序，GPA最高的学生将获得排名1。

Windowing Functions 窗口函数

窗口函数用于在查询结果集的特定“窗口”内执行计算。窗口函数允许我们在不聚合数据的情况下计算累计和、移动平均值等。

```
select account_number, date_time,  
       sum (value) over  
         (partition by account_number  
          order by date_time  
          rows unbounded preceding)  
       as balance  
from transaction  
order by account_number, date_time
```

- 该查询计算每个账户的累计余额，按日期时间排序。
- sum() 函数在每个账户的交易窗口内计算累计和。